

TruCart

AI-Assisted E-Commerce Operations Platform

Full Project Report – Architecture, Agents, APIs, Infra & Interview Notes

Bhautik Vaghamshi

Daksh Patel

B.Tech Semester 7 – Hackathon Build

September 15, 2026

What this document is. A complete, plain-language walkthrough of everything that is actually implemented in the TruCart codebase – not the future vision, not the pitch deck. Every section maps to real files in the repository so you can open the code and match it line-for-line while explaining it to an interviewer. Diagrams are hand-drawn (TikZ) to mirror the Mermaid diagrams already in `docs/architecture.md`.

Contents

1	The Problem, In One Paragraph	2
2	System Architecture	2
2.1	Why this shape	2
3	The Six Agents	3
3.1	Guardrail thresholds – where they live	5
4	Multi-Agent Coordination	6
5	The Autopilot Ledger (Headline Feature)	6
5.1	How one decision is booked	6
5.2	Baseline formulas (the actual math)	7
5.3	Verification and Trust Score	7
5.4	Standing policies – learning from human rejections	8
5.5	Fail-safe design	8
6	Database	8
7	APIs	9
7.1	FastAPI (<code>backend/</code>) – does the real work	9
7.2	Next.js route handlers (<code>apps/web/app/api/</code>) – thin auth-gated proxies	9
7.3	Server actions – not REST at all	9
8	Frontend / Dashboard	10
9	Authentication & Security	10
10	Docker & Deployment	11
10.1	Two containers	11
10.2	Two compose files, two purposes	11
10.3	AWS EC2 deployment (the live demo)	11

11 CI Pipeline	12
12 LLM Integration & Observability	12
13 Autonomy Modes	13
14 Scope & Honest Limitations	13
15 Interview Q&A Cheat Sheet	14

1 The Problem, In One Paragraph

Running an e-commerce store means juggling six back-office jobs at once: watching stock levels, confirming and shipping orders, answering support tickets, repricing products as costs and demand shift, running clearance campaigns on overstock, and tracking shipments. Normally a person (or a team) does each of these jobs by hand, in separate tools, which means decisions are slow and inconsistent. **TruCart replaces each of those six jobs with a small, specialised AI agent** that watches its own slice of the database, makes a bounded decision (with an LLM as an advisor, never as an unchecked authority), executes the decision itself if it is low-risk, and pushes it to a human *review queue* if it is not. A scheduler runs every agent in sequence every few minutes so the store keeps running on its own; a human only steps in for the judgement calls that matter.

The distinguishing feature – and the one worth leading with in an interview – is the **Autopilot Ledger**: every autonomous decision is booked against a computed “what would have happened if we did nothing” baseline, checked against the real outcome weeks later, and rolled into a trust score per agent. This turns “the AI made a decision” into “the AI made a decision, and here is the receipt proving whether it was a good one.”

2 System Architecture

TruCart is a small monorepo (npm workspaces + Turborepo) with three real pieces: a Next.js dashboard, a FastAPI agent service, and a Postgres (Supabase) database. Nothing else is a separate service – there is no message queue, no container orchestrator, no external microservice mesh. Simplicity was a deliberate choice: every piece needed to run for \$0 on free tiers.

Layer	Technology	Where it runs
Dashboard	Next.js 16 (App Router), React 19, Tailwind v4, shadcn/ui (packages/ui)	Vercel Hobby (or Docker)
Web APIs / auth	Next.js route handlers, jose HS256 JWT session cookie	same as above
Agents / orchestrator	FastAPI, APScheduler, OpenAI-compatible SDK	Fly.io / Render / EC2 / Docker
Database	Supabase Postgres + pgvector, plain SQL migrations (no ORM)	Supabase free tier
LLM inference	Local Ollama qwen2.5:7b or any OpenAI-compatible free endpoint (Groq, OpenRouter)	self-host or free API
Tracing	Langfuse (langfuse.openai drop-in wrapper)	Langfuse Cloud free

2.1 Why this shape

- **No ORM.** All 16 migrations are hand-written SQL. This keeps every RPC (stored procedure) auditable and lets the team encode business rules (e.g. “never let a reservation exceed stock on hand”) directly in the database as atomic transactions, rather than in application code that could race.
- **No message bus.** Agents coordinate purely through shared Postgres state and a single `correlation.id` per cycle – see §4. This was a conscious trade-off: much simpler to reason about and debug than a pub/sub system, at the cost of only supporting sequential hand-offs.
- **LLM is advisory, not authoritative.** Every agent calls the LLM for one bounded JSON decision (e.g. “raise, reduce, or hold, and to what price”), then clamps that decision with

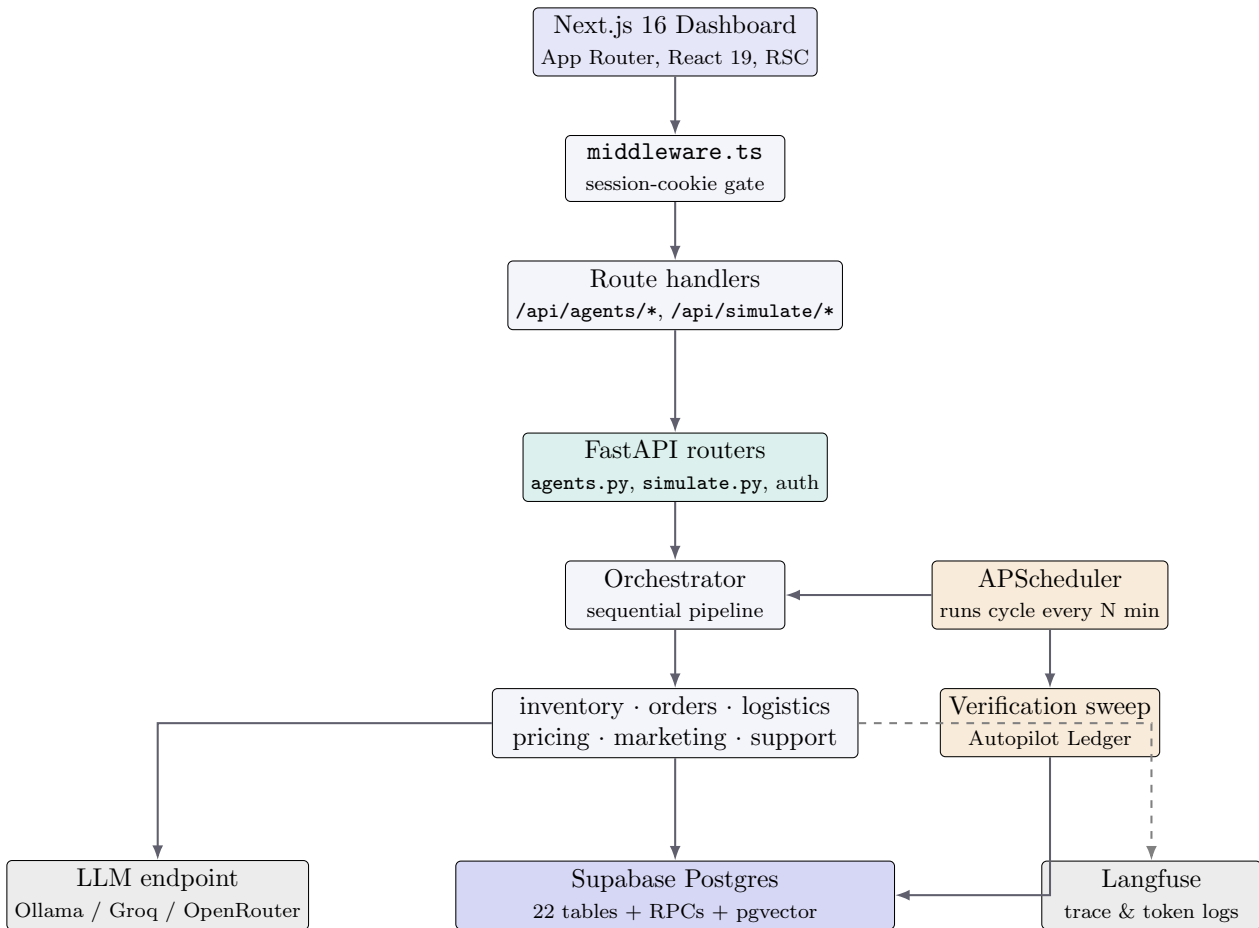


Figure 1: Three real tiers: browser → Next.js web tier → FastAPI agent tier → Postgres, with an LLM and Langfuse as optional side dependencies.

deterministic guardrails written in Python. If the LLM is unreachable, the agent falls back to a deterministic rule and keeps working – the system never breaks because an LLM timed out.

3 The Six Agents

All agents live in `backend/agents/` and share one shape: they load their own tunable knobs from the `agent_config` table, scan their slice of the database, optionally ask the LLM for a judgement call via `call_llm_json(...)`, apply hard guardrails, execute the safe actions immediately, push the risky ones to `review_queue`, write one row to `agent_task_log`, and emit `notify(...)` rows that show up as dashboard notifications.

1. Inventory Agent (`inventory.py`)

Restocking – finds products at or below their reorder point and raises purchase orders.

- Scans `products` joined with `inventory` for items where `quantity_on_hand ≤ reorder_point`.
- Orders a fixed `reorder_quantity` (not demand-forecasted in this build – a known simplification, see §14).
- The LLM writes the human-readable restock justification text only; the reorder decision itself is rule-based.
- **Guardrail:** a PO auto-approves only if its total cost $\leq$ `po_auto_approve_limit` (default

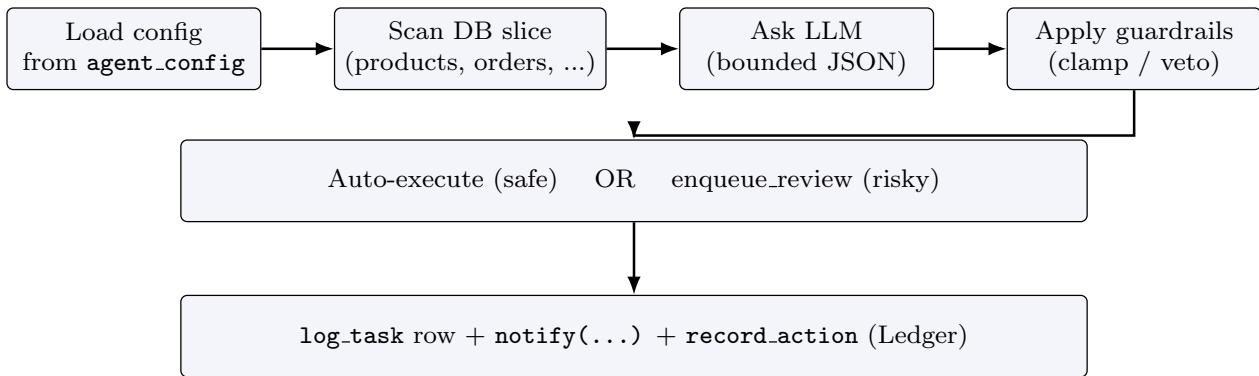


Figure 2: The common shape every agent follows.

Rs. 5,000); otherwise it goes to `review_queue`.

- **Receiving:** approved POs are received automatically after `receiving_lead_days` via the `receive_purchase_order` RPC, which increments on-hand stock – so the loop (order → receive → stock goes up) is closed, not just the ordering half.

2. Order Agent (`orders.py`)

Confirms paid orders and reserves stock – purely mechanical, no LLM.

- Finds pending orders with `payment_status = 'paid'`.
- Calls the `confirm_order_and_reserve` RPC, an atomic Postgres transaction that checks stock availability and reserves it in the same statement – preventing a race where two concurrent orders both think stock is available.
- **Escalates** orders that are unpaid, have no items, or exceed available stock.

3. Logistics Agent (`logistics.py`)

Drives the shipment lifecycle – purely mechanical, no LLM.

- Creates a shipment for every newly confirmed order (invents a carrier name, tracking number, and flat shipping cost – there is no real carrier API).
- Advances shipments through `label_created` → `in_transit` → `out_for_delivery` → `delivered` on a time basis (`transit_hours`), not real carrier webhooks.
- Flags shipments stuck at one stage too long as exceptions for review.

4. Pricing Agent (`pricing.py`)

The most sophisticated agent – reprices products based on our own economics, not competitor scraping.

- For every active product it computes: the **effective unit cost** (from the latest approved/received purchase order, falling back to `cost_price`), the resulting **current gross margin**, **days of stock cover** (on-hand stock ÷ trailing 90-day daily sales velocity), and days since the last price change.
- Flags a candidate if margin is *squeezed* below `min_margin_pct`, cost *rose* enough to threaten the target margin, the product is *overstocked* (cover > `overstock_cover_days` and margin already healthy), or cost *fell*.

- Sends the top candidates to the LLM with exactly these numbers and asks for **raise|reduce|hold**, a target price, urgency, and a one-sentence rationale.
- **Guardrails clamp the LLM’s answer no matter what it said:** the final price is capped to `±price.change_max_pct` (15%) of the current price, floored at the margin that yields `min_margin_pct`, and **never allowed below unit cost**.
- **Escalates** if the move would go below cost, breach the margin floor, or the LLM marked it `urgency: high`. Applied through the `apply_price_change` RPC (which records `price_history` and updates `products.current_price` atomically).

5. Marketing Agent (`marketing.py`)

Detects overstock and drafts clearance campaigns.

- Overstock test: `on_hand > reorder_point × a multiplier` (a coarse heuristic, not true demand modelling).
- The LLM writes the campaign subject, body copy, and target segment.
- **Guardrail:** auto-activates if the estimated budget $\leq$ `budget_auto_approve_limit`; otherwise goes to review.
- “Sending” a campaign just inserts a `campaigns` row and fabricated metrics – there is no real ESP integration.
- **Conflict guard:** the orchestrator passes marketing a `skip_product_ids` set containing every SKU that inventory or pricing already touched *in the same cycle*, so marketing never launches a clearance sale on a product that was simultaneously repriced or just reordered.

6. Support Agent (`support.py`)

Triages open tickets with retrieval-augmented generation (RAG).

- Retrieves the customer’s order history and, via the `match_knowledge_base` pgvector RPC, the top relevant policy/FAQ chunks (return policy, refund windows, etc.).
- The LLM **chooses the action** – resolve, refund, or escalate – with a confidence score, using the retrieved context.
- **Guardrail:** a refund amount is clamped to the order total and auto-approves only under `refund_auto_approve_limit` (default Rs.100); larger refunds always go to review. Refunds are whole-order only (no partial refunds / RMA in this build).

3.1 Guardrail thresholds – where they live

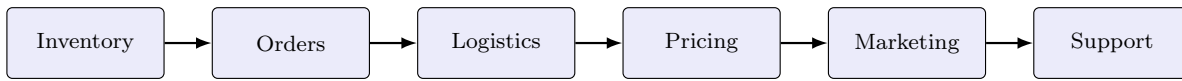
Two config tables separate “how does this agent behave” from “what dollar amount needs a human”:

- `store_config` – cross-cutting approval ceilings: `po_auto_approve_limit`, `refund_auto_approve_limit`, `price_change_max_pct`, plus the Ledger’s settle-day windows.
- `agent_config` – per-agent tunables: `max_items_per_run`, `min_margin_pct`, `target_margin_pct`, `demand_window_days`, etc.

Both are editable live from `/dashboard/agents` → **Configure** – no redeploy needed to tighten or loosen an agent’s autonomy.

4 Multi-Agent Coordination

TruCart deliberately does **not** use a message bus, an event queue, or agent-to-agent messaging. Coordination is **sequential + shared database state**:



All six run under one shared `correlation_id` per cycle. One agent's failure is caught and logged; the cycle continues with the rest (`status`: completed / partial / error).

Figure 3: `orchestrator.py` – a static pipeline, not a graph. This is the whole of `run_orchestrator()`.

- `orchestrator.py` holds a static `_PIPELINE` list and calls each `run_<name>_agent(correlation_id)` once, in the fixed order `inventory → orders → logistics → pricing → marketing → support`.
- Agents hand off work **through the database**, never directly: inventory writes purchase orders that pricing later reads as its cost basis; the order agent flips an order to **confirmed**, which the logistics agent then picks up to create a shipment.
- The **human hand-off** works the same way: an agent calls `enqueue_review` which inserts a `review_queue` row; a human resolves it on `/dashboard/review`, which calls the server action `updateReviewStatus(...)` – this cascades the decision into the right domain table (the PO, the price, the order, the refund, the campaign, the shipment) and stamps the reviewer's id and note.
- `correlation_id` is a **grouping key**, not a channel – it lets you look at `agent_task_log` or a Langfuse trace and see everything that happened in one cycle, but no agent ever reads another agent's `correlation_id` to react to it.
- A **conflict guard** (`touched_product_ids`) is the one piece of real cross-agent signalling within a cycle: it is accumulated in memory as the pipeline runs and passed to marketing so it does not double-act on a SKU another agent already changed.

Interview angle: “why not a real multi-agent framework (LangGraph, CrewAI)?”

Honest answer: for six agents with a fixed, known execution order and no need for dynamic re-planning, a static pipeline plus shared Postgres state is simpler to debug, has zero extra infrastructure, and every hand-off is auditable as ordinary rows in a table. A graph-based orchestrator would earn its complexity if agents needed to dynamically decide *who runs next* based on results – that is explicitly named as future work, not something avoided out of ignorance.

5 The Autopilot Ledger (Headline Feature)

This is the feature to lead with. Most “agentic AI” demos show an agent making a decision and stop there – there is no way to know if the decision was actually *good*. The Autopilot Ledger closes that loop for every judgement call the pricing, inventory, marketing, and support agents make.

5.1 How one decision is booked

Every time an agent auto-executes or escalates a decision, it calls `record_action(...)` in `backend/agents/ledger` which inserts one `agent_action` row containing:

1. **The decision itself** – e.g. `{old_price, new_price, effective_cost}`.
2. **A counterfactual baseline** – the Rs. outcome of *doing nothing*, computed right now from data the agent already has (trailing sales velocity, margins, stock cover). The formula and every input value are stored in the row, so any number on the dashboard can be traced back to exactly how it was derived.
3. **verify_after** – a settle window per action type, read from `store_config` (e.g. 14 days for a price change, 7 + lead time for a purchase order, 30 days for a refund).

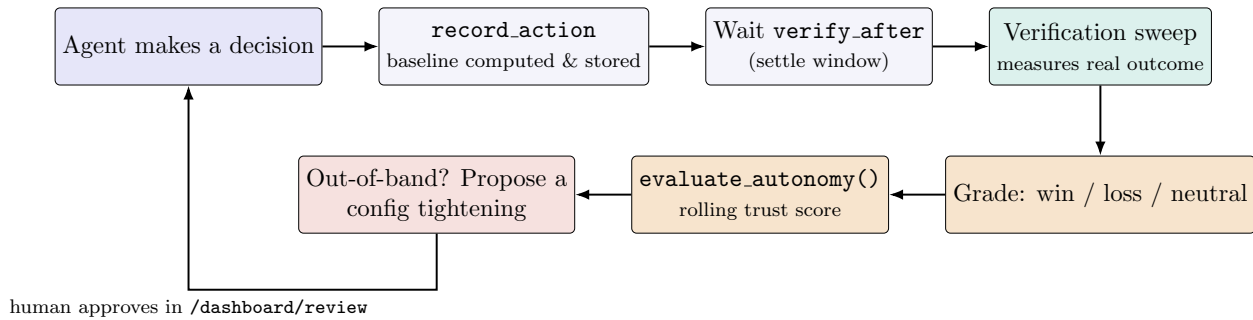


Figure 4: The Ledger’s full lifecycle: decide → baseline → settle → verify → grade → trust score → (optionally) propose tightening the agent’s own limits.

5.2 Baseline formulas (the actual math)

Action	Baseline (“do nothing”) formula
Price change	<code>daily_velocity * settle_days * (old_price - effective_cost)</code> – the margin dollars we’d have kept selling at the old price.
Purchase order	<code>-(max(0, velocity*lead_days - available) * margin_per_unit)</code> – the margin we’d have <i>lost</i> from stockouts if we hadn’t reordered, net of a holding-cost charge on the PO’s value.
Campaign	0 (no campaign = no lift); projected value is an assumed $1.5\times$ ROAS on budget – explicitly labelled <i>estimated</i> , not measured, because there’s no real ESP.
Refund	<code>-support_handle_cost_inr</code> – the labour cost of escalating instead of resolving now; also labelled <i>estimated</i> .

5.3 Verification and Trust Score

- `backend/agents/verification.py run_verification_sweep()` runs as a second scheduled job (or on-demand via `POST /api/ledger/verify`). For every `agent_action` whose `verify_after` has passed, it re-reads the operational tables (actual sales, actual stock, actual refund status) to compute `realized_delta_inr`, compares it to `baseline_delta_inr`, and grades the decision `win / loss / neutral`.
- `evaluate_autonomy()` then looks at an agent’s last `ledger_min_sample` verified decisions. If the win-rate falls outside the `[ledger_win_rate_floor, ledger_win_rate_ceiling]` band, it enqueues a `review_queue` item of type `autonomy_adjustment` that proposes a **concrete, one-click config change** (e.g. “`budget_auto_approve_limit 200 → 120`”), linked to the exact losing decisions that justify it.

- Approving that proposal in `/dashboard/review` writes the new `store_config/agent_config` value directly – **the system tightens its own leash based on evidence**, instead of a human guessing at a new threshold.
- `/dashboard/ledger` shows, per agent: a **Trust Score**, realized Rs. contribution (with a measured-vs-estimated split clearly labelled), and a drill-down into every decision’s stored baseline formula and actual outcome.

5.4 Standing policies – learning from human rejections

When an operator rejects a review item, they can attach a one-line standing rule (stored in `agent_policy`). `base.py`’s `load_active_policies(agent_name)` prepends every active rule for that agent to its LLM system prompt on *every future run* – so “don’t discount SKU X below Rs. 500” said once keeps being obeyed.

5.5 Fail-safe design

`record_action` is **best-effort**: any failure (including the Ledger tables not existing yet, i.e. migration `016_ledger.sql` not applied) is caught and logged, never raised. The six agents run identically with or without the Ledger – it observes, it never gates.

6 Database

Supabase Postgres, 22 tables, all schema and seed data as plain numbered SQL migrations in `database/migrations/` (001–016) – no ORM, no migration framework magic, just files run in order.

Migration	What it adds
001	Extensions (<code>pgvector</code>) + a generic <code>updated_at</code> trigger function
002–004	Core schema: foundation tables, transactional tables (orders, order_items, purchase_orders...), operational tables (inventory, shipments, tickets...)
005	Seed data (demo store, users, products)
006	Row-Level Security policies
007	<code>notifications</code> table (dashboard feed)
008, 010, 014	<code>agent_config</code> reseeds / updates as tunables evolved
009	Fixes broken <code>updated_at</code> triggers
011	Prunes seeded <code>agent_task_log</code> noise
012	<code>match_knowledge_base</code> <code>pgvector</code> similarity RPC (support RAG)
013	Agent RPCs: <code>apply_price_change</code> , <code>create_po_with_review</code> , <code>confirm_order_and_reserve</code>
015	Pricing demo signal seed data
016	Autopilot Ledger : <code>agent_action</code> , <code>agent_policy</code> tables + <code>receive_purchase_order</code> / <code>release_order_reservation</code> RPCs

Why RPCs (stored procedures) for the risky writes: `apply_price_change`, `confirm_order_and_reserve`, and `receive_purchase_order` all touch two or more tables that must stay consistent (e.g. reserve stock *and* confirm the order, or nothing). Doing that as a single Postgres function call makes it one atomic transaction instead of two separate round-trips from Python that could interleave with another request.

7 APIs

There are two API surfaces, deliberately kept thin.

7.1 FastAPI (backend/) – does the real work

Method	Path	Purpose
GET	/, /health	Liveness check
POST	/api/auth/login	Verifies bcrypt/SHA1 password, 5-per-15-min rate limit per email+IP. Returns the user; does not issue a token – the web layer signs the session.
POST	/api/auth/change-password	Re-verifies current password before writing a new bcrypt hash
POST	/api/agents/{agent_name}/run	Runs one agent or the orchestrator; returns {status, log_id, correlation_id, summary: {scanned, auto_executed, escalated}}
GET	/api/agents/scheduler	{enabled, running, interval_minutes, last_cycle_at, last_cycle_status} for the “Autonomous mode” badge
POST	/api/simulate/orders	Inserts realistic synthetic orders (?count, ?trigger_agents) – this is how demo data flows in, since there is no real storefront
GET	/api/ledger/summary	Ledger report cards per agent: wins/losses, realized Rs., trust score
GET	/api/ledger/actions	Raw agent_action rows for the drill-down table
POST	/api/ledger/verify	Runs the verification sweep on demand

Interactive Swagger docs are served live at /docs (and ReDoc at /redoc) – worth opening in a live demo.

7.2 Next.js route handlers (apps/web/app/api/) – thin auth-gated proxies

Every route here does one job: check the `trucart_session` cookie (via `middleware.ts`), then forward the request to FastAPI. `/api/auth/login` is the one exception – on a successful FastAPI response it *signs* the HS256 session cookie (8-hour expiry, `httpOnly`). `/api/auth/logout` just clears it.

7.3 Server actions – not REST at all

Most dashboard mutations (updating a review, changing a config value, editing a profile) never go through an HTTP API. They are Next.js **server actions** defined in `apps/web/app/dashboard/actions.ts`, called directly from React components, running on the server with the Supabase service-role client. The most important one, `updateReviewStatus(reviewId, status, {note, makeStandingRule})`, resolves the reviewer from the verified session, cascades the approval into the right domain table, and (if the item is an `autonomy_adjustment`) writes the proposed config change.

Interview angle: “REST API vs. server actions – why both?”

Server actions exist because the dashboard reads/writes are same-origin, same-request mutations triggered from a form or button – a full REST round trip would just be overhead. The FastAPI surface exists because agent execution is genuinely a different service (Python, long-running LLM calls, its own deploy target) that the Next.js app must call over the network. Using server actions everywhere would have meant putting agent logic inside Next.js; keeping FastAPI separate keeps the Python ML/agent code independently deployable and testable.

8 Frontend / Dashboard

Next.js 16 App Router, React 19 Server Components, Tailwind v4, and a shared `shadcn/ui` component package (`packages/ui`) used by both `apps/web` and any future app in the monorepo. Screens live one-per-folder under `apps/web/app/dashboard/`:

Route	What it shows
<code>/dashboard</code>	Overview – KPIs, recent activity
<code>/dashboard/agents</code>	Run agents on-demand, Configure knobs, Autonomous-mode badge
<code>/dashboard/orders</code>	Order list, “Simulate incoming orders” button
<code>/dashboard/inventory</code>	Stock levels, purchase orders
<code>/dashboard/logistics</code>	Shipment lifecycle tracking
<code>/dashboard/pricing</code>	Price history, current margins
<code>/dashboard/marketing</code>	Campaigns
<code>/dashboard/support</code>	Ticket queue
<code>/dashboard/review</code>	Human review queue – approve/reject every escalation
<code>/dashboard/ledger</code>	Autopilot Ledger – trust scores, realized Rs., drill-down
<code>/dashboard/audits</code>	<code>agent_task_log</code> history per cycle
<code>/dashboard/notifications</code>	Live notification feed
<code>/dashboard/settings</code>	Profile, password, account deactivation

Pattern used throughout: each screen is a pair of files – `page.tsx` is a Server Component that fetches data directly from Supabase (no client-side loading spinners for the initial view) and `*-client.tsx` holds the interactive parts (tables with filters, buttons that call server actions). This keeps data fetching co-located with rendering and avoids waterfalls of client-side `fetch` calls.

9 Authentication & Security

- Single staff role model (`users.role = 'admin'`) – this is an internal ops tool, not a customer-facing app, so there is no signup flow or multi-tenant RBAC.
- **Login flow:** browser → Next.js `/api/auth/login` → FastAPI verifies the password hash (bcrypt, with a legacy SHA1 path) and applies an in-memory rate limit (5 attempts / 15 minutes per email+IP) → on success, the **Next.js layer** signs an 8-hour HS256 JWT into an `httpOnly trucart_session` cookie. FastAPI itself never issues a token.
- `middleware.ts` verifies that cookie on every request to `/dashboard/*`, `/api/agents/*`, and `/api/simulate/*`; unauthenticated API calls get 401, pages redirect to `/login`.
- The cookie is only marked **Secure** when the request actually arrived over HTTPS (checked via `x-forwarded-proto`) – a small but important detail that lets login work over plain HTTP on an EC2 demo box without a TLS-terminating proxy in front, without weakening security on a real HTTPS deployment.

- FastAPI holds the Supabase **service-role** key (bypasses Row-Level Security) – by design it must sit behind the Next.js/Vercel proxy and never be exposed directly to the public internet in a real deployment.
- Account management: `POST /api/auth/change-password` re-verifies the current password; `deactivateAccount` sets `users.is_active = false`.
- Known gaps, stated openly: no signup, no MFA, no server-side session revocation list (the JWT is valid until it expires, full stop).

10 Docker & Deployment

10.1 Two containers

Image	How it's built
backend/Dockerfile	<code>python:3.12-slim</code> → installs build tools (for <code>bcrypt</code> 's native extension) → <code>pip install -r requirements.txt</code> → copies <code>backend/</code> → runs <code>uvicorn backend.main:app</code> on port 8000.
apps/web/Dockerfile	Multi-stage: <code>deps</code> (installs <code>npm</code> workspace deps), <code>builder</code> (bakes <code>NEXT_PUBLIC_API_URL</code> and <code>SESSION_SECRET</code> as build args, runs <code>next build</code> with Next's <i>standalone</i> output), <code>runner</code> (non-root <code>nextjs</code> user, copies only the standalone bundle + static assets, runs <code>node server.js</code> on port 3000).

Prebuilt images are published to Docker Hub as `bhautik2004/trucart-backend` and `bhautik2004/trucart-web`.

10.2 Two compose files, two purposes

- `docker-compose.yml` (local dev) – **builds** both images from source, wires `web`'s `NEXT_PUBLIC_API_URL` build-arg to `http://backend:8000` (the Docker-internal service name, not `localhost`, because the variable gets baked into client-shipped JS at build time), exposes `web` on port 80 and `backend` on 8000.
- `docker-compose.prod.yml` (deployment) – **pulls** the prebuilt Docker Hub images instead of building on the target machine. This matters concretely: a `t2.micro` EC2 instance (1 vCPU / 1 GB RAM) cannot reliably run `next build`, but running the two already-built containers fits comfortably (roughly 150 MB combined).

10.3 AWS EC2 deployment (the live demo)

1. Launch a `t2.micro/t3.micro` instance (Amazon Linux 2023, free tier), using `deploy/ec2-bootstrap.sh` as the instance *User data* script – it installs Docker + Compose and clones the repo automatically at boot.
2. Open inbound ports 80 (dashboard) and 8000 (API) in the security group.
3. Write real secrets into `~/TruCart/.env.local` on the instance (Supabase keys, an LLM API key, a real `SESSION_SECRET`), plus `CORS_ALLOW_ORIGINS=http://<EC2-PUBLIC-IP>`.
4. `docker compose -f docker-compose.prod.yml --env-file .env.local pull && up -d`.

To ship a code change: rebuild + push the image(s) *locally*, then on the instance just re-run `pull && up -d` – no build ever happens on the tiny instance itself. On the live box, `OLLAMA_BASE_URL`

is pointed at Groq’s OpenAI-compatible endpoint (a free hosted LLM) instead of a local Ollama server, since a 7B model won’t fit in 1 GB of RAM.

11 CI Pipeline

`.github/workflows/ci.yml` runs on every push to `main` and every pull request, as two independent jobs on `ubuntu-latest`:

Job	Steps
backend	Checkout → set up Python 3.12 → <code>pip install -r backend/requirements-dev.txt</code> → <code>python -m pytest backend/tests -q</code>
web	Checkout → set up Node 20 (npm cache) → <code>npm ci</code> → <code>npx turbo lint typecheck build --filter=web --filter=@workspace/ui</code> (with a placeholder <code>SESSION_SECRET</code> so the build doesn’t fail on the startup guard)

This is intentionally a **quality gate, not a deploy pipeline** – there is no CD step; deployment is the manual Docker Hub push + EC2 pull described above. Turborepo’s `--filter` flags mean only the `web` app and the shared `ui` package are linted/typechecked/built (the two packages that matter for a passing frontend build), keyed to Turborepo’s own dependency graph.

Interview angle: “why no CD / auto-deploy?”

Time-boxed hackathon scope: a manual two-command deploy (`pull && up -d`) to a single EC2 box is proportionate to a single-environment demo. The natural next step named in the docs is a GitHub Actions job that builds & pushes the Docker images on merge to `main`, and optionally SSHes into the instance to pull – straightforward to add on top of the existing Dockerfiles without changing them.

12 LLM Integration & Observability

- All agents talk to the LLM through one function: `call_llm_json(system_prompt, user_prompt, ...)` in `base.py`. It always asks for `response_format={"type": "json_object"}` at low temperature (0.2), parses the result, and returns `None` on *any* failure – malformed JSON, network error, timeout – so the calling agent can fall back to a deterministic rule instead of crashing.
- The client is pointed at an OpenAI-compatible base URL (`OLLAMA_BASE_URL`), which is why the same code path works against local Ollama, Groq, or OpenRouter – just swap the base URL, model name, and API key.
- **Langfuse** is wired in as a drop-in replacement for the OpenAI client (`langfuse.openai`) when its keys are configured – every chat completion is automatically captured as a trace with model, token usage, latency, and errors, with *zero* manual span code in the agents themselves. Each trace’s id is deterministically derived from the same `correlation_id` already written to `agent_task_log`, so a Supabase row and a Langfuse trace can be cross-referenced by one id.
- Tracing is **fully best-effort**: if Langfuse keys are missing or its SDK throws during setup, tracing is silently and permanently disabled for the rest of the process, and the underlying LLM call still happens normally.

- Token usage per agent run is accumulated across every LLM call an agent makes during one `correlation_id`, then drained into the single `agent_task_log.tokens_used` column when that agent finishes – so “run full cycle” shows one token count per agent, not one per LLM call.

13 Autonomy Modes

- **Manual:** “Run Agent” / “Run full cycle” buttons on `/dashboard/agents` call the route handler, which calls the FastAPI runner directly.
- **Scheduled:** `backend/scheduler.py` starts an `APScheduler BackgroundScheduler` when `SCHEDULER_ENABLED=true`, running `run_orchestrator` every `ORCHESTRATOR_INTERVAL.MINUTES` (default 10). This is literally what “autonomous” means in this build – a timer, not event-driven triggering.
- A **second** scheduled job runs the Ledger’s verification sweep independently of the main pipeline.
- Every runner is **idempotent**: it skips items already queued, POs already open, or products with a too-recent price change, and is bounded by `max_items_per_run` – so running the same cycle twice in a row (or the scheduler firing while a manual run is still warm) never double-executes anything.

14 Scope & Honest Limitations

Stated openly in the README – know these cold, because an interviewer probing depth will ask about them and the strongest answer is to name the simplification *before* they do.

- **No real storefront.** Orders arrive via `POST /api/simulate/orders` – there’s no check-out, payment gateway, or fraud check to integrate with.
- **No carrier API.** Logistics invents carrier/tracking/cost and advances shipments on a timer, not real carrier webhooks.
- **Inventory reorders a fixed quantity** – not demand-forecasted; it does receive POs and increment stock, but does not update `cost_price` on receipt.
- **Marketing overstock detection is a coarse ratio test**, and “sending” a campaign just inserts a row – no ESP, fabricated metrics.
- **Support refunds are whole-order only**, no payment gateway, no partial refund/RMA.
- **No outbound customer comms** – the notification feed is internal-only.
- **Ledger’s marketing/support baselines are modelled, not measured** – clearly labelled “estimated” wherever shown, versus pricing/inventory which use real operational data (“measured”).
- **RAG needs a one-off embedding backfill** (`npm run seed:rag`) before the support agent has any knowledge-base context; without it, it triages blind.
- **Auth has no MFA, no signup, no session revocation store.**

The large AWS/Bedrock/LangGraph design in `ai-ecommerce-platform-plan.md` is the *funded production target*, explicitly documented as **not** what runs here – everything in this report is what is deployable on free tiers today.

15 Interview Q&A Cheat Sheet

“Walk me through what happens when the scheduler fires.”

APScheduler calls `run_orchestrator()` with a fresh `correlation_id`. It runs inventory, then orders, then logistics, then pricing, then marketing, then support, each writing its own `agent_task_log` row under that id. Each agent loads its config, scans its slice of the DB, optionally asks the LLM for a judgement call, clamps that with guardrails, executes safe actions immediately via RPCs, and enqueues anything risky to `review_queue` (which also creates a dashboard notification). Every judgement-call decision (pricing/inventory/marketing/support) also gets booked into the Autopilot Ledger with a baseline. If any one agent throws, it's caught, logged as an error for that agent, and the rest of the pipeline still runs.

“How do you prevent the LLM from doing something dangerous?”

The LLM never executes anything directly – it only returns a JSON opinion (e.g. target price, raise/reduce/hold, urgency). Python code then clamps that opinion against deterministic guardrails computed independently of the LLM: a price can never go below unit cost or move more than 15% in one step; a refund can never exceed the order total or auto-approve above Rs.100; a purchase order over Rs.5,000 always needs a human. If the LLM is unreachable or returns garbage, `call_llm_json` returns `None` and the agent falls back to a pure rule-based decision (usually: escalate).

“What’s the hardest technical problem you solved?”

Making the reservation/confirmation path race-safe without a lock service: two orders for the same last unit of stock must not both succeed. Solved with the `confirm_order_and_reserve` Postgres RPC – a single atomic transaction that checks availability and reserves stock in one statement, so Postgres’s own transaction isolation does the concurrency control instead of application-level locking.

“How do you know the agents are actually helping, not just automating noise?”

That’s exactly what the Autopilot Ledger measures. Every decision is booked against a computed counterfactual baseline at decision time, then graded weeks later against what actually happened. The per-agent trust score and realized-Rs. numbers on `/dashboard/ledger` come directly from that, and if an agent’s win-rate falls outside an acceptable band the system proposes tightening its own auto-approve limit – with the losing decisions attached as evidence – rather than a human guessing.

“Why FastAPI + Next.js instead of one framework end to end?”

The agent/orchestrator logic is Python because that’s where the LLM/data-science ecosystem lives and because it needed to be independently deployable/scalable/schedulable (APScheduler) from the web tier. The dashboard is Next.js because Server Components let most screens fetch directly from Supabase with zero client-side loading state, and `shadcn/ui` gives a fast, consistent component set. The two talk over a small, explicit REST surface plus a session cookie – not tightly coupled, easy to reason about, easy to deploy separately (Vercel + any Python host).

“What would you build next if you had more time?”

In order of value: (1) event-driven triggering instead of a fixed timer (DB triggers/webhooks so agents react within seconds, not up to 10 minutes); (2) demand-forecasted reorder quantities instead of a fixed number; (3) a real ESP and carrier API integration behind the same guardrail pattern already in place; (4) partial refunds/RMA; (5) a CD step that builds and pushes the

Docker images automatically on merge to **main**.

Generated as an interview preparation reference from the TruCart codebase.
Source: docs/, README.md, backend/agents/, apps/web/, .github/workflows/ci.yml.